9.2. Konfliktus sorbarendezhetőség

Most egy olyan elégséges feltételt adunk meg, mely biztosítja egy ütemezés sorbarendezhetőségét. A piaci rendszerek ütemezői a tranzakciók sorbarendezhetőségére általában ezt az erősebb feltételt biztosítják, amelyet „konfliktus sorbarendezhetőségnek” nevezünk. Ez a *konfliktus* (conflict) fogalmon alapul: amely olyan egymást követő műveletpár az ütemezésben, amelynek ha a sorrendjét felcseréljük, akkor legalább az egyik tranzakció viselkedése megváltozhat.

9.2.1. Konfliktusok

Azzal kezdjük, hogy vegyük észre a legtöbb műveletpár *nincs* konfliktusban a fenti értelemben. Ugyanis, tételezzük fel, hogy T_i és T_j különböző tranzakciók, vagyis $i \neq j$.

1. $r_i(X)$; $r_j(Y)$ sohasem konfliktus, még akkor sem, ha $X = Y$. Ennek az az oka, hogy egyik lépés sem változtatja meg az értékeket.
2. $r_i(X)$; $w_j(Y)$ nincs konfliktusban, feltéve, ha $X \neq Y$. Ennek az az oka, hogy a T_j írhatja az Y-t, mielőtt a T_i beolvasta az X-et, az X értéke ugyanis ettől nem változik. Annak sincs hatása a T_j-re, hogy a T_i olvassa az X-et, ugyanis ez nincs hatással arra, hogy milyen értéket ír be a T_j az Y-ba.
3. $w_i(X)$; $r_j(Y)$ nincs konfliktusban, ha $X \neq Y$, ugyanazért, mint a 2.
4. Szintén hasonlóan $w_i(X)$; $w_j(Y)$ sincs konfliktusban mindaddig, amíg $X \neq Y$.

Másrészt három esetben nem cserélhetjük fel a műveletek sorrendjét:

a) Ugyanannak a tranzakciónak két művelete, pl. $r_i(X)$; $w_i(Y)$ konfliktus. Ennek az az oka, hogy egyetlen tranzakción belül a műveletek sorrendje rögzített, és az adatbázis-kezelő rendszer ezt a sorrendet nem rendezheti át újra.
b) Különböző tranzakciók ugyanarra az adatbáziselemre való írása konfliktus. Vagyis $w_i(X)$; $w_j(X)$ konfliktus. Ennek az az oka, mint már írtuk, hogy az X értéke az marad, amelyet a T_j számolt ki. Ha felcseréljük a sorrendjüket, hogy $w_j(X)$; $w_i(X)$, akkor az X-nek a T_i által kiszámított értéke marad meg. Az a feltevésünk, hogy „nincs egybeesés”, azt adja, hogy a T_i és a T_j által írt értékek lehetnek különbözőek, és ezért az adatbázis valamelyik kezdeti állapotára különbözni fognak.
c) Különböző tranzakcióknak ugyanabból az adatbáziselemből való olvasása és írása is konfliktus. Vagyis $r_i(X)$; $w_j(X)$ konfliktus, és $w_i(X)$; $r_j(X)$ is konfliktus. Ha átvisszük $w_j(X)$-et $r_i(X)$ elé, akkor a T_i által olvasott X-beli érték az lesz, amelyet a T_j írt, amiről pedig feltételeztük, hogy nem szükségképpen egyezik meg az X korábbi értékével. Tehát $r_i(X)$ és $w_j(X)$ sorrendjének cseréje befolyásolja, hogy T_i milyen értéket olvas X-ből, ez pedig befolyásolja a T_i működését.

Levonhatjuk a következtetést, hogy különböző tranzakciók bármely két műveletének sorrendje felcserélhető, hacsak nem

1. Ugyanarra az adatbáziselemre vonatkoznak, és
2. Legalább az egyik művelet írás.

Ezt az elvet kiterjesztve tetszőleges ütemezést véve annyi nem konfliktusos cserét készíthetünk, amennyit csak kívánunk, abból a célból, hogy az ütemezést soros ütemezéssé alakítsuk át. Ha ezt meg tudjuk tenni, akkor az eredeti ütemezés sorba rendezhető volt, ugyanis az adatbázis állapotára való hatása változatlan marad minden nem konfliktusos cserével.

Azt mondjuk, hogy két ütemezés *konfliktus ekvivalens* (conflict-equivalent), ha szomszédos műveletek nem konfliktusos cseréinek sorozatával az egyiket átalakíthatjuk a másikká. Azt mondjuk, hogy egy ütemezés *konfliktus sorbarendezhető* (conflict-serializable schedule), ha konfliktus ekvivalens valamely soros ütemezéssel. Megjegyezzük, hogy a konfliktus sorbarendezhetőség elégséges feltétele a sorbarendezhetőségnek, vagyis egy konfliktus sorbarendezhető ütemezés sorba rendezhető ütemezés is egyben. Azonban a konfliktus-sorbarendezhetőség nem szükséges ahhoz, hogy egy ütemezés sorba rendezhető legyen, mégis általában ezt a feltételt ellenőrzik a piaci rendszerek ütemezői, amikor a sorbarendezhetőséget kell biztosítaniuk.

9.6. példa: Legyen az ütemezés

$r_1(A)$; $w_1(A)$; $r_2(A)$; $w_2(A)$; $r_1(B)$; $w_1(B)$; $r_2(B)$; $w_2(B)$;

a 9.5. példából. Azt állítjuk, hogy ez az ütemezés konfliktus sorbarendezhető. A 9.8. ábrán látható a cserék sorozata, amellyel ez az ütemezés átalakítható a (T_1, T_2) soros ütemezéssé, ahol az összes T_1-beli művelet megelőzi az összes T_2-beli műveletet. Aláhúztuk azokat a szomszédos műveletpárokat, amelyeket felcserélünk az egyes lépésekben. •

$r_1(A)$; $w_1(A)$; $r_2(A)$; $\underline{w_2(A);\ r_1(B)}$; $w_1(B)$; $r_2(B)$; $w_2(B)$;
$r_1(A)$; $w_1(A)$; $\underline{r_2(A);\ r_1(B);}$ $w_2(A)$; $w_1(B)$; $r_2(B)$; $w_2(B)$;
$r_1(A)$; $w_1(A)$; $r_1(B)$; $r_2(A)$; $\underline{w_2(A);\ w_1(B)}$; $r_2(B)$; $w_2(B)$;
$r_1(A)$; $w_1(A)$; $r_1(B)$; $\underline{r_2(A);\ w_1(B);}$ $w_2(A)$; $r_2(B)$; $w_2(B)$;
$r_1(A)$; $w_1(A)$; $r_1(B)$; $w_1(B)$; $r_2(A)$; $w_2(A)$; $r_2(B)$; $w_2(B)$;

9.8. ábra. *Egy konfliktus sorbarendezhető ütemezés szomszédos műveletek felcserélésével való átalakítása soros ütemezéssé*

9.2.2. Megelőzési gráfok és teszt a konfliktus sorbarendezhetőségre

Viszonylag könnyű megvizsgálnunk egy S ütemezést, és eldöntenünk, hogy konfliktus sorbarendezhető-e vagy nem. Az az alapötlet, hogy ha valahol konfliktusban álló műveletek szerepelnek az S-ben, ezeket a műveleteket végrehajtó tranzakcióknak ugyanabban a sorrendben kell előfordulniuk a konfliktus ekvivalens soros ütemezésekben, mint ahogyan az S-ben voltak. Tehát a konfliktusban álló műveletpárok megszorítást adnak a feltételezett konfliktus ekvivalens soros ütemezésben a tranzakciók sorrendjére. Ha ezek a megszorítások nem mondanak egymásnak ellent, akkor találhatunk konfliktus ekvivalens soros ütemezést. Ha pedig ellentmondanak egymásnak, akkor tudjuk, hogy nincs ilyen soros ütemezés.

Adott a T_1 és T_2 tranzakcióknak, esetleg további tranzakcióknak, egy S ütemezése. Azt mondjuk, hogy T_1 *megelőzi* T_2-t, és $T_1 <_S T_2$-vel jelöljük, ha van a T_1-ben olyan A_1 művelet, és a T_2-ben olyan A_2, hogy

1. A_1 megelőzi A_2-t az S-ben,
2. A_1 és A_2 ugyanarra az adatbáziselemre vonatkoznak, és
3. A_1 és A_2 közül legalább az egyik írás művelet.

Megjegyezzük, hogy ezek pontosan azok a feltételek, amikor nem lehet felcserélni az A_1 és A_2 sorrendjét. Tehát, A_1 az A_2 előtt szerepel bármely az S-sel konfliktus ekvivalens ütemezésben. Ennek eredményeként, ha ezek közül az ütemezések közül az egyik soros ütemezés, akkor ebben T_1-nek meg kell előznie T_2-t.

Ezeket a megelőzéseket a *megelőzési gráfban* (precedence graph) összegezhetjük. A megelőzési gráf csomópontjai az S ütemezés tranzakciói. Ha a tranzakciókat T_i-vel jelöljük az i függvényében, akkor a T_i-nek megfelelő csomópontot az i egésszel jelöljük. Az i csomópontból a j csomópontba vezet irányított él, ha $T_i <_S T_j$.

9.7. példa: A következő S ütemezés a T_1, T_2, T_3 három tranzakciót tartalmazza:

S: $r_2(A)$; $r_1(B)$; $w_2(A)$; $r_3(A)$; $w_1(B)$; $w_3(A)$; $r_2(B)$; $w_2(B)$;

Ha az A-val kapcsolatos műveleteket nézzük meg, akkor több okot találunk, hogy miért igaz a $T_2 <_S T_3$. Például $r_2(A)$ az S-ben $w_3(A)$ előtt áll, és $w_2(A)$ az $r_3(A)$ és a $w_3(A)$ előtt is áll. A három észrevételünk közül bármelyik elegendő, hogy igazoljuk, valóban vezet él a 2-ből 3-ba a 9.9. ábrán szereplő megelőzési gráfban.



9.9. ábra. *A 9.7. példa S ütemezéséhez tartozó megelőzési gráf*

Hasonló módon ha megnézzük a B-vel kapcsolatos műveleteket, akkor szintén több okot találunk, hogy miért igaz a $T_1 <_S T_2$. Például $r_1(B)$ művelet a $w_2(B)$ művelet előtt áll. Tehát az S megelőzési gráfjában az 1-ből 2-be szintén vezet él. Tulajdonképpen ezek és csak ezek az élek azok, amelyeket az S ütemezésben szereplő műveletek sorrendjéből tudunk ellenőrizni. •

Van egy egyszerű szabály, amivel megmondhatjuk, hogy egy S ütemezés konfliktus sorbarendezhető-e:

Miért nem szükséges konfliktus sorbarendezhetőség a sorbarendezhetőséghez?

Egy példát már láttunk a 9.7. ábrán. Akkor megnéztük, hogy a T_2 által végrehajtott speciális számítások miatt hogyan vált az ütemezés sorba rendezhetővé. Pedig a 9.7. ábra ütemezése nem konfliktus sorbarendezhető, ugyanis az *A*-t T_1 írja előbb, a *B*-t pedig a T_2. Mivel sem az *A* írását, sem a *B* írását nem lehet átrendezni, semmilyen módon nem kerülhet T_1 összes művelete a T_2 összes művelete elé, sem fordítva.

Azonban vannak olyan sorba rendezhető, de nem konfliktus sorbarendezhető ütemezések is, amelyek nem függnek a tranzakciók által végrehajtott számításoktól. Például tekintsük a T_1, T_2, és T_3 három tranzakciót, amelyek mindegyike *X* értékét írja. A T_1 és T_2 az *Y*-nak is ír értéket, mielőtt az *X*-nek írnának értéket. Az egyik lehetséges ütemezés, amely itt éppen soros is, a következő:

S_1: $w_1(Y)$; $w_1(X)$; $w_2(Y)$; $w_2(X)$; $w_3(X)$.

Az S_1 ütemezés *X* értékének a T_3 által írt értéket, *Y* értékének pedig a T_2 által írt értéket adja. Ugyanezt végzi a következő ütemezés is:

S_2: $w_1(Y)$; $w_2(Y)$; $w_2(X)$; $w_1(X)$; $w_3(X)$.

Intuíció alapján átgondolva annak, hogy a T_1 és a T_2 milyen értéket ír az *X*-be, nincs hatása, ugyanis a T_3 felülírja az értékeket. Emiatt S_1 és S_2 az *X*-nek is és az *Y*-nak is ugyanazt az értéket adja. Mivel az S_1 soros, és az S_2-nek bármely adatbázis-állapotra ugyanaz a hatása, mint az S_1-nek, tehát S_2 sorba rendezhető. Ugyanakkor mivel nem tudjuk felcserélni $w_1(Y)$-t $w_2(Y)$-nal, és nem tudjuk felcserélni $w_1(X)$-et $w_2(X)$-szel, így cseréken keresztül nem lehet az S_2-t valamelyik soros ütemezéssé átalakítani. Tehát S_2 sorba rendezhető, de nem konfliktus sorbarendezhető.

- Rajzoljuk fel az *S* megelőzési gráfját, és nézzük meg tartalmaz-e kört!

Ha igen, akkor *S* nem konfliktus sorbarendezhető. Ha pedig a gráf körmentes, akkor *S* konfliktus sorbarendezhető, továbbá a csomópontok bármelyik topologikus sorrendje[1] megadja a konfliktus ekvivalens soros sorrendet.

9.8. példa: A 9.9. ábra megelőzési gráfja körmentes, így a 9.7. példa *S* ütemezése konfliktus-sorbarendezhető. A csomópontoknak, azaz a tranzakcióknak csak egyetlen sorrendje van, amely konzisztens a gráf éleivel, és ez: (T_1, T_2, T_3). Megjegyezzük, hogy az *S*-et valóban át lehet alakítani olyan ütemezéssé, amelyben a három tranzakció mindegyikének az összes művelete ugyanebben a sorrendben van, és ez a soros ütemezés:

S'': $r_1(B)$; $w_1(B)$; $r_2(A)$; $w_2(A)$; $r_2(B)$; $w_2(B)$; $r_3(A)$; $w_3(A)$;

Ahhoz, hogy belássuk, megkaphatjuk az *S*-ből az S'-t szomszédos elemek cseréjével, az első észrevételünk, hogy az $r_1(B)$-t konfliktus nélkül az $r_2(A)$ elé hozhatjuk. Ezután három cserével a $w_1(B)$-t közvetlenül az $r_1(B)$ utánra tudjuk cserélni, ugyanis mindegyik közbeeső művelet az *A*-ra vonatkozik, és a *B*-re nem. Ezután az $r_2(B)$-t és a $w_2(B)$-t csak az *A*-ra vonatkozó műveleteken keresztül át tudjuk vinni pontosan a $w_2(A)$ utáni helyzetbe, amivel megkapjuk az S'-t. •

[1] Egy körmentes gráf *topologikus sorrendje* a csomópontok bármely olyan rendezése, amelyben minden $a \rightarrow b$ élre, az *a* csomópont megelőzi a *b* csomópontot a topologikus rendezésben. Bármely körmentes gráfnak találhatunk topologikus rendezettséget úgy, hogy többszörösen ismételve töröljük azokat a csomópontokat, amelyeknek nincs megelőzője a maradék csomópontok között.

9.9. példa: Tekintsük az alábbi ütemezést:

S_1: $r_2(A)$; $r_1(B)$; $w_2(A)$; $r_2(B)$; $r_3(A)$; $w_1(B)$; $w_3(A)$; $w_2(B)$;

amely csak abban különbözik az *S*-től, hogy az $r_2(B)$ művelet három hellyel előbb szerepel. Az *A*-ra vonatkozó műveleteket megvizsgálva most is csak a $T_2 <_{S_1} T_3$ megelőzési kapcsolathoz jutunk. De, ha a *B*-t vizsgáljuk, akkor nemcsak $T_1 <_{S_1} T_2$ teljesül (ugyanis $r_1(B)$ és $w_1(B)$ a $w_2(B)$ előtt szerepel), hanem $T_2 <_{S_1} T_1$ is (ugyanis $r_2(B)$ a $w_1(B)$ előtt fordul elő). Emiatt az S_1 ütemezéshez tartozó megelőzési gráf az, amely a 9.10. ábrán látható.

9.10. ábra. *A 9.9. példa S_1 ütemezéséhez tartozó ciklikus megelőzési gráf, ez az ütemezés nem konfliktus sorbarendezhető*

Ez a gráf nyilvánvalóan tartalmaz kört. Arra következtethetünk, hogy S_1 nem konfliktus sorbarendezhető, ugyanis intuíció alapján láthatjuk, hogy bármely konfliktus ekvivalens soros ütemezésben a T_1-nek T_2 előtt is és után is kellene állnia, így emiatt nem létezik ilyen ütemezés. •

9.2.3. Miért működik a megelőzési gráfon alapuló tesztelés?

Láttuk, hogy a megelőzési gráfban a kör túl sok megszorítást jelent a feltételezett konfliktusekvivalens soros ütemezésre nézve. Azaz, ha létezik $T_1 \rightarrow T_2 \rightarrow \ldots \rightarrow T_n \rightarrow T_1$ *n* darab tranzakcióból álló kör, akkor a feltételezett soros sorrendben T_1 műveleteinek meg kell előznie a T_2-ben szereplő műveleteket, amelyek megelőzik a T_3-belieket és így tovább egészen T_n-ig. De a T_n műveletei emiatt a T_1-beliek mögött vannak, ugyanakkor meg is kellene előzniük a T_1-belieket a $T_n \rightarrow T_1$ él miatt. Ebből következik, hogyha a megelőzési gráf tartalmaz kört, akkor az ütemezés nem konfliktus-sorbarendezhető.

A másik irányt egy kicsit nehezebb belátnunk. Azt kell megmutatnunk, hogy amikor a megelőzési gráf körmentes, akkor az ütemezés műveletei átrendezhetők szomszédos műveletek szabályos cseréivel úgy, hogy az ütemezés egy soros ütemezéssé váljon. Ha ezt meg tudjuk tenni, akkor bebizonyítottuk, hogy minden körmentes megelőzési gráffal rendelkező ütemezés egyben konfliktus-sorbarendezhető. A bizonyítás az ütemezésben részt vevő tranzakciók száma szerinti indukcióval történik.

Alapeset: Ha $n = 1$, vagyis csak egyetlen tranzakcióból áll az ütemezés, akkor az ütemezés már önmagában soros, emiatt biztosan konfliktus-sorbarendezhető.

Indukció: Legyen *S*

$T_1, T_2, \ldots, T_n$

n darab tranzakció műveleteiből álló ütemezés. Tételezzük fel, hogy *S*-nek körmentes megelőzési gráfja van. Ha a véges gráf körmentes, akkor van legalább egy olyan csomópontja, amelybe nem vezet él. Legyen a T_i tranzakciónak megfelelő *i* csomópont egy ilyen csomópont. Mivel nem vezet az *i* csomópontba él, nincs az *S*-ben olyan *A* művelet, hogy:

1. Valamelyik T_j, a T_i-től különböző tranzakcióra vonatkozzon,
2. T_i valamely műveletét megelőzi, és
3. Ezzel a művelettel konfliktusban van.

Ugyanis, ha lenne ilyen, akkor a megelőzési gráfban be kellene rajzolnunk egy élt a *j* csomópontból az *i* csomópontba.

Így lehetséges, hogy a T_i minden műveletét az *S* legelejére mozgatjuk át, miközben megtartjuk a sorrendjüket. Az ütemezés most a következő alakú:

(T_i műveletei) (a többi $n - 1$ tranzakció műveletei)

Most tekintsük az *S* második részét, vagyis a T_i-től különböző összes tranzakciónak a műveleteit. Mivel ezek a műveletek egymáshoz viszonyítva ugyanabban a sorrendben vannak, mint ahogyan az *S*-ben voltak, ennek a második résznek a megelőzési gráfja megegyezik az *S* olyan megelőzési gráfjával, amelyből elhagyjuk a T_i

csomópontot és az ebből a csomópontból kimenő éleket.

Mivel az eredeti megelőzési gráf körmentes volt, és csomópontok, illetve élek törlésével nem válhatott ciklikussá, következik, hogy a második rész megelőzési gráfja is körmentes. Továbbá, mivel a második része $n - 1$ tranzakciót tartalmaz, alkalmazzuk rá az indukciós feltevést. Így tudjuk, hogy a második rész műveletei szomszédos műveletek szabályos cseréivel átrendezhetők soros ütemezéssé. Ily módon magát az S-et alakítottuk át olyan soros ütemezéssé, amelyben a T_i műveletei állnak legelöl, és a többi tranzakció műveletei ezután következnek valamilyen soros sorrendben. Az indukciót beláttuk, és így következik, hogy minden olyan ütemezés, amelynek körmentes a megelőzési gráfja, egyben konfliktus-sorbarendezhető.